

PaCR: An Agentic Preference-aligned TAP Conflict Handling

Ronghua Li, Zi Liang, Yaxin Xiao, Qingqing Ye, *Member, IEEE*, Haibo Hu, *Senior Member, IEEE*

Abstract—With the increasing complexity of smart home environments, Trigger-Action-Programming (TAP) conflicts have emerged as critical issues, occurring when multiple automation apps compete for control over shared devices or interact in unintended ways. While existing approaches detect and resolve such conflicts using pre-defined rules or expert knowledge, they fail to account for the significant variability in individual preferences for conflict resolution and significantly degrade automation system utility. To address this gap, we propose PaCR (Preference-aligned Conflict Resolver), a novel agentic framework that resolves conflicts while aligning with users' personalized preferences for automation behavior. PaCR employs a two-step conflict resolution process: first, it interactively gathers contextual information from smart home configurations to generate conflict resolution agents with user profiles that accurately depict user preferences and characteristics; then, the agents acting as users utilize semantic reasoning capabilities of LLMs to assist in generating personalized conflict detection and resolution strategies. Due to the agentic conflict resolving feature, we evaluate the effectiveness of PaCR with existing datasets at scale. PaCR demonstrates strong capabilities in addressing conflicts based on users' need and significantly improves conflict resolution efficacy and interpretability, offering a user-centric approach to managing smart home automation conflicts.

Index Terms—Smart Home automation, automation conflict, rule modification, personal variance alignment, end-user interaction

I. INTRODUCTION

With the rapid advancement of Internet of Things (IoT) technologies, smart systems have proliferated across diverse environments. A smart home serves as a representative example of such sophisticated systems, operating through centralized platforms that integrate various IoT devices, including environmental sensors, actuators, and AI assistants. These systems are designed to enhance user convenience via automation mechanisms, such as the scheduled deployment of robotic vacuums for household maintenance or the implementation of automated shutdown protocols for air conditioners to optimize energy efficiency. To extend these capabilities, leading smart home platforms now host dedicated automation marketplaces [1], [2], [3], [4] featuring curated official applications.

Nonetheless, despite the operational conveniences of smart home configurations, emerging research has uncovered critical vulnerabilities in device-automation interdependencies [5], [6],

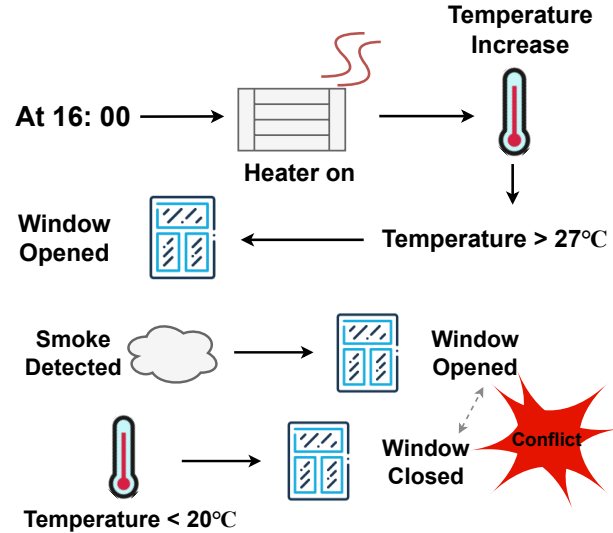


Fig. 1: Examples of interaction threats and conflicts. **Interaction:** A time-triggered app turns on the heater, which increases indoor temperature and interacts with another app that opens the window [5]; **Conflict:** An app opens the window for smoke evacuation, but another app closes the window due to a temperature drop.

[7], [8], [9]. As shown in Fig. 1, a heating unit scheduled to activate at 16:00 may elevate ambient temperatures above 27°C, which could inadvertently trigger secondary trigger-action programs (TAPs, or IFTTT-style **apps**) that initiate window ventilation protocols. Alternatively, smoke evacuation applications may conflict with thermal comfort maintenance systems due to overlapping window operations. Such unintended interaction chains can ultimately drive smart environments into states that contradict users' original automation intentions.

In response to these identified challenges, existing studies [6], [10], [11], [5], [7], [12], [13], [14] have primarily focused on mitigating these vulnerabilities through centralized analysis. Nevertheless, current research paradigms overlook users' proclivity to create custom automation apps [15], which exhibit broader functional diversity and heterogeneous design rationales. This oversight fundamentally limits the efficacy of existing systems [6], [10], [11], [5] in three critical dimensions:

First, contemporary conflict detection systems exhibit limited efficacy when processing apps with divergent design objectives. Current methodologies [7], [6], [12], [13], [14] employ rule-based detection heuristics predicated on predefined risk indicators. However, the determination of undesirable system states constitutes a subjective judgment that exhibits significant contextual variance across users and environmental

Manuscript received April 19, 2021; revised August 16, 2021.

Ronghua Li, Zi Liang, Yaxin Xiao, Qingqing Ye are with the Department of Electrical and Electronic Engineering, The Hong Kong Polytechnic University, Hong Kong; (e-mail: cory-ronghua.li@connect.polyu.hk; zi1415926.liang@connect.polyu.hk; 20034165r@connect.polyu.hk; qqing.ye@polyu.edu.hk)

Haibo Hu is with the Department of Electrical and Electronic Engineering, The Hong Kong Polytechnic University, Hong Kong and PolyU Research Centre for Privacy and Security Technologies in Future Smart Systems. (e-mail: haibo.hu@polyu.edu.hk;)

conditions. For instance, window-opening automation may represent either a security vulnerability or a benign operation, depending on whether occupants are present.

Second, most existing systems focus on conflicts between two apps, ignoring complex device-app or human-app interactions. For example, an app that opens windows for ventilation may conflict with a manual operation that closes windows for security reasons. Current detection frameworks [8], [6], [10], [5] do not account for such user-initiated actions, leading to incomplete conflict identification.

Third, predominant approaches lack comprehensive static conflict resolution (SCR) frameworks that balance adaptability with feasibility. Effective SCR requires providing prescriptive guidance for automation modification. However, current implementations demonstrate fundamental limitations: some merely flag conflicts without actionable modification strategies [16], [17], while others propose overly simplistic modification results [7], [6], [18], [12] that neglect user preference heterogeneity. For example, existing systems may recommend complete app removal [11] without considering functionality preservation requirements.

To address these limitations, we present PaCR (Preference-aligned Conflict Resolution), an LLM-agentic framework that iteratively refines automation apps while resolving conflicts through preference-aware optimization. Specifically, this work proposes the first **Agentic Conflict Handling framework**. While LLMs demonstrate strong capabilities, their direct application to conflict resolution tasks remains challenging [19], [20]. To address this, we implement a stepwise interaction process for agents: (i) It first analyzes existing smart home configurations and queries a simulated "user" agent to learn their design rationales and smart home preferences; (ii) The system then applies a specialized agentic workflow to leverage these insights for conflict detection and resolution.

Our solution presents two fundamental innovations: (1) We propose a structured preference context that captures user expectations through preference articulation, enabling personalized conflict mitigation with minimal degradation in utility; (2) We integrate conflict-resolving agents into pipelines, harnessing their semantic reasoning capabilities for device-interaction analysis and context-aware solution generation. This significantly reduces the cognitive burden on users when acknowledging conflicts and implementing resolutions.

The rest of the paper is organized as follows. In Section II, we introduce the background, large language models, and definitions of conflicts. In Section III, we analyze the advantages and disadvantages of related work. Section IV presents our rationale for resolving conflicts in a personalized manner and details the design of PaCR. We evaluate PaCR's system performance in Section V and conduct relevant discussions in Section VI.

II. BACKGROUND

A. Smart Home Automation

Smart home technology has evolved rapidly over the past few decades, transforming modern living environments through centralized platforms [2], [4], [3]. These platforms

interconnect IoT devices, enabling users to monitor device states and remotely control actuators. Users can configure trigger-action programs (TAPs/IFTTT rules) to automate device operations based on predefined conditions (e.g., activating air conditioning when the temperature exceeds specified thresholds). Each automation app consists of three components: (1) a trigger event, (2) conditional checks, and (3) executable actions. When the triggered conditions are met, the app executes the designated operations. Throughout this work, we use the term "**app**" to refer to these automation rules consistently.

B. Large Language Model

The advent of ChatGPT [21] has driven the widespread adoption of large language models (LLMs). Trained on extensive datasets, these models exhibit strong capabilities in contextual understanding and response generation. To utilize LLMs effectively, structured prompts must be designed using techniques such as contextual framing, few-shot examples, and chain-of-thought reasoning [22], [23], [24]. Our framework leverages these prompt engineering methods to harness LLMs' semantic understanding for analyzing app-device interactions and identifying potential conflicts.

C. Conflict Definitions

We define conflicts as undesirable behaviors of smart devices caused by interactions between automation logic and devices. These include: **I. Unwanted competing control over the same devices (Device Control Conflict, DCC)**; **II. Counteracting or mitigating influences on physical environments (Property Influence Conflict, PIC)**; and **III. Inappropriate interactions among automation rules that contradict users' original expectations (Interaction Source Conflict, ISC)**. These three high-level conflict types encompass all previously identified conflicts [10], [6], [8] and additionally include a new category of user-app conflicts. We elaborate on the detailed definition of each conflict type in Section IV-E. Furthermore, since intra-app logical errors have been extensively studied [25], [26], [6], this work focuses on the detection and reasoning of inter-app conflicts, thereby complementing existing intra-app conflict resolution approaches.

III. RELATED WORK

Automation conflicts have emerged from research on smart home security. Studies by [6], [5] first identified a new type of threat caused by conflicts or interactions between automation rules. This threat occurs when two apps compete for control over the same device, or when the functionality of one app unexpectedly interacts with the triggers, conditions, or actions of another app. Following this research trajectory, [10], [14], [13], [11], [27], [9], [8] have proposed both static and dynamic approaches to detect and resolve such conflicts. While this work shares similar conflict detection and resolution objectives, we aim to address several persisting limitations, as outlined below.

A. Conflict Detection

Previous conflict detection systems rely on either offline static analysis or online dynamic analysis. Static conflict detection (SCD) systems generally use natural language processing (e.g., embedding similarity) and other software algorithms to analyze relationships between devices and apps [7], [16], [6], [5], [12], but they suffer from high false-positive rates due to inaccurate semantic analysis when using naive models such as Word2Vec [28]. For example, a plug indicator light might be incorrectly associated with illuminance or brightness. To address this issue, [10], [13] proposed dynamic conflict detection (DCD) to validate static analysis results in real-world environments. However, despite its accuracy, this approach has fundamental limitations. First, it is time-consuming (e.g., [13] reports an average detection time of two hours) and inevitably disrupts users' routines by requiring repeated device cycling. Second, dynamic analysis provides only one-time validation, while environmental conditions like temperature and humidity evolve continuously. This temporal variability can alter detection accuracy over time. Some works, such as those by Xing et al. [29], [30], employ large language models (LLMs) to label relationships and construct graph neural networks (GNNs) that capture conflicting relationships between components. A key limitation of these works, however, is that conflicts labeled by LLMs often deviate from human perception—raising questions about their real-world utility and effectiveness. Therefore, training or fine-tuning a model to address this discrepancy can easily reduce its generality and adaptability to most users.

To achieve reliable conflict detection while maintaining efficiency, this work adopts static analysis enhanced with LLMs to reduce false positives. As demonstrated by [31], LLM-based prompt engineering with contextual embeddings effectively resolves semantic inaccuracies, significantly improving the performance of static analysis methods [5], [11], [12]. Additionally, we incorporate user preferences to ensure the authenticity of detected conflicts.

B. Conflict Resolution

Conflict resolution methods are equally important as detection and are also categorized into dynamic and static approaches. Dynamic methods typically use third-party hardware hubs or cloud servers to mediate communication between devices and platforms during runtime. However, most vendors implement closed-source systems, which limits cross-platform adaptability [13], [14]. IoTMediator [10] connects devices to a centralized hub that simulates virtual devices to enable platform integration. Nevertheless, its hardware requirement creates deployment barriers for average users. Static approaches offer broader adaptability by providing automation modification suggestions (e.g., adding or removing conditions) to preemptively mitigate risks. This enables conflict management even in closed-source platforms. Despite their cross-platform advantages, existing static solutions [5], [11], [6] generate generic modifications that disregard user personalization, or delete risky actions entirely [12]—reducing system utility [11]. Furthermore, involving users in the resolution step introduces additional uncontrollable variance, as users

themselves are prone to making errors. To our knowledge, no current static method effectively resolves conflicts while preserving functionality. Moreover, requiring users to identify conflicting scenarios also imposes significant cognitive burden.

C. Language Models and Agents for Smart Homes

Smart home automation generation has emerged as a critical research area for enhancing system utility and user experience. Studies by [26], [32], [33], [34], [35], [36], [37], [38], [39], [40], [41] have explored translating user descriptions or event traces into structured automation rules. However, training specialized models for this task requires substantial resources. Recent trends instead utilize pre-trained LLMs with prompt engineering [21], [42], [43]—an approach aligned with our use of LLMs' reasoning capabilities for conflict resolution.

In the context of smart home service provision, we also observe the rise of LLM-powered agents [44], [45], [46] that autonomously perform tasks through iterative reasoning and tool usage. Works such as [47], [48] apply LLM agents for natural language-based smart home control. However, these systems primarily focus on executing direct user commands rather than resolving conflicts. Our work pioneers the integration of LLM agents into a structured workflow specifically designed for personalized conflict detection and resolution in smart home environments.

D. Beyond Automation Conflicts

IoT systems also present numerous security and privacy challenges that require investigation. Works by [49], [50], [51] focus on detecting device anomalies and alerting users to unexpected scenarios. Other studies, such as [52], [53], [54], [55], [56], address personal data leakage and develop privacy-preserving solutions for modern IoT systems.

IV. SYSTEM DESIGNS

A. Design Rationale

We use a toy survey example to illustrate why user preference is important for personalized conflict detection and resolution. Suppose that APP1 “opens the window when smoke is detected” and APP2 “closes the window when the temperature drops below 26 degrees Celsius.” When smoke is detected and the outdoor temperature drops, potential conflicts arise regarding control over the window. However, after surveying people offline with dozens of such seemingly conflicting pairs, we surprisingly noticed that some people consider the situations to be conflicting, while others do not care which event happens first, demonstrating significant variance in conflict interpretation and handling preferences. Due to such variance, we intend to incorporate users' own preferences into the system to ensure that the conflict handling results do not degrade system utility. Yet, there are several challenges to achieving such goals:

First, we find that users exhibit difficulties in comprehending potential conflicts, even if they are explained in natural language. Second, representing user preferences in a structured manner for conflict detection and resolution is non-trivial.

Users may struggle to articulate their expectations in a way that can be directly utilized by automated systems.

To this end, this work proposes an agentic refinement and conflict-resolving workflow to help users handle conflicts automatically. The core idea is to build an agent that can represent users to detect conflicts and address them in a nearly personalized manner. In the following sections, we first introduce the core steps of data processing, agent construction, and conflict handling. Additionally, since the agentic workflow demands a large amount of prompt engineering, we further introduce how we leverage meta-prompting techniques in Section IV-F to automatically refine all prompts for the agent and enhance the overall output quality.

B. System Overview

In this section, we propose PaCR, a **P**reference-aligned **C**onflict **R**esolution framework (Fig. 2). It comprises three core steps: Knowledge Preparation (KP), Preference Enumeration (PE), and Conflict Handling (CH). In the first step (**KP**), PaCR analyzes the semantic contexts of smart home configurations (e.g., automation apps and device lists) (①) and portrays users via profiles, i.e., their living habits and characteristics (②). The main goal of KP is to build the knowledge base for the Conflict Resolving Agents (CRA). Since the automatically generated profiles may have only captured general characteristics, in the second step (**PE**), the CRA further enumerates potential user preferences for each of their automation apps to complement the comprehension of user preferences and rationales (③). Finally, in **CH**, the CRA leverages all app preference information to detect conflicts, address them, and apply fixing configurations to the smart home setting (④). In the following, we introduce the detailed designs of each step.

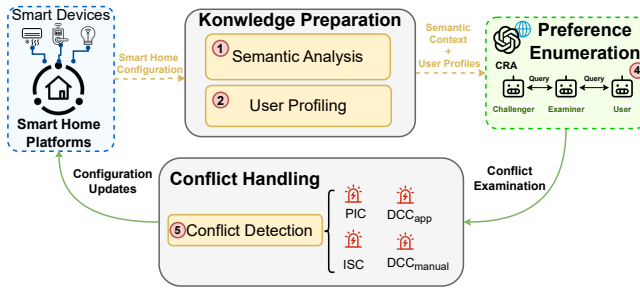


Fig. 2: Overview of PaCR framework.

C. Knowledge Preparation

KP serves two purposes: First, it collects basic information, such as the list of devices and existing automation apps, into the system and infers the functions of devices to assist subsequent inter-device/app interaction analysis. Second, it deduces the design rationales behind each app to profile users from the perspective of their smart home usage habits and preferences.

To begin with, PaCR analyzes the nature of devices, their capabilities, and their potential relationships with the environment or other devices, namely, Semantic Relationship Analysis

(SRA). SRA aims to examine the relationships between these devices and 20 common physical properties¹, as shown in Fig. 3. However, as discussed in Section III, most existing SRA implementations rely on Word2Vec or n-gram models, which suffer from a high false positive rate. In addition, most LLM-assisted methods rely on comprehensive device descriptions from users, such as user manuals, which is very inconvenient for older devices. Given the recent proliferation of large language models (LLMs) and their enhanced reasoning capabilities, PaCR adopts LLMs for SRA, using purely device names and related automation apps. We first tested the baseline LLM-SRA, i.e., directly asking LLMs whether a device is related to another device over certain properties, and encountered a problem: the limited device configuration information (e.g., only device names/types) restricts the LLM's ability to fully comprehend device capabilities, potentially generating erroneous relationship analyses. To address this limitation, PaCR leverages a two step search: (i) it searches the device on Tasmota [57], an open-sourced IoT firmware platform that supports hundreds of IoT devices, for the sensing/actuation capabilities of the device. (ii) If exact match is not found, PaCR prompts LLMs to interpret function lists of similar devices, thereby enriching contextual information for subsequent analysis. In addition, all automation apps related to the device are also provided as context for the models to better understand their working scenarios and capabilities. Such enriched context improves the LLM's capacity to reason about device-property interactions. After employing Chain-of-Thought [23] and few-shot learning [22] techniques, hallucinations [58] in SRA barely existed during the experiments. To ensure the correctness of SRA results, PaCR further applies consistency check to ensure that all semantic relationships at least relate to one similar device in the database.

Next, as shown in Fig. 3, we formalize SRA results into intermediate data structures on a device-basis, similar to the "Scene Interaction" reference in [59]: each device will be linked to a set of properties that it interferes with or senses. During the subsequent conflict reasoning process, only entities with intersecting properties will be interpreted as "interactable and potentially conflicting" and processed to reduce computational overhead and improve accuracy.

After obtaining contextual information via SRA, PaCR uses LLMs to compile a comprehensive user profile for each app set, aiming to depict users from the perspective of design rationales, personal characteristics, and their potential interaction scenarios with the smart home environment. For instance, if a user has multiple apps related to temperature control, we may infer that the user is sensitive to temperature changes and prefers a comfortable indoor climate. This also applies to other aspects, such as security consciousness (e.g., multiple security-related apps), energy-saving preferences (e.g., apps related to power consumption), and convenience-oriented behaviors (e.g., apps automating routine tasks). By integrating these insights, PaCR constructs a user profile that encapsulates

¹Properties are summarized from previous works, including *sound, temperature, gas, smoke, humidity, illuminance, air quality, carbon dioxide, infrared, ultraviolet, dust, power, energy, water, human voice, weather, acceleration, contact, presence, motion*.

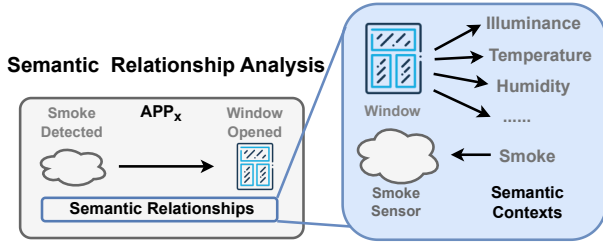


Fig. 3: SRA infers apps' and devices' semantic contexts with environmental properties.

user preferences and characteristics, serving as the knowledge foundation for the preference-aligned conflict resolution agent (CRA). While this cannot ensure a complete representation of user preferences, KP provides options for real users to review and adjust the generated profiles, thereby refining the system's understanding based on actual user feedback.

In summary, KP builds the knowledge foundation for the CRA by collecting smart home configurations, analyzing device-app-environment relationships, and profiling users based on their app design rationales and preferences. Such information is essential for the CRA to understand user preferences and effectively detect and resolve conflicts in subsequent steps.

D. Preference Enumeration

In the preference enumeration step, PaCR further strengthens the knowledge foundation for each automation app, i.e., *why each app is created*. More specifically, the strengthening process includes reasoning about why/when the app activates, how it influences the smart home, and when it should cease activation. Referencing these three aspects, PaCR implements more accurate conditions for activation, ensures the proper functionality of apps, and avoids unexpected condition enabling/disabling. However, since cultivating such information can be cognitively demanding for users, the CRA creates three sub-agents—the *Examiner*, the *User*, and the *Challenger*—to enumerate on these three perspectives. The idea is intuitive: the *User* refers to the user profile obtained from KP to fully represent the user; the *Examiner* asks the *User* questions to understand the design rationales behind each app; and the *Challenger* challenges the *User* with counterfactual scenarios to further explore potential missing conditions or unexpected interactions. The overall question framework is shown in Fig. 4, which includes three questions:

- **Direct Q:** How will the app be used? [*User* → *Examiner*]
- **Counterfactual Q1:** What if the app is triggered/conditioned by [*Challenger* examples]? [*Challenger* → *User* → *Examiner*]
- **Counterfactual Q2:** What if [*Challenger* examples] change the action states? [*Challenger* → *User* → *Examiner*]

Based on the dialogue contexts, the CRA summarizes the user's preference/intentions into a structured data piece as

shown in Listing 1. Correspondingly, each part of the context is used to address one of the three types of conflict: the Activation context for ISC; the Functioning context for PIC; and the Deactivation context for DCC. In the following, we introduce the detailed construction process for the preference context.

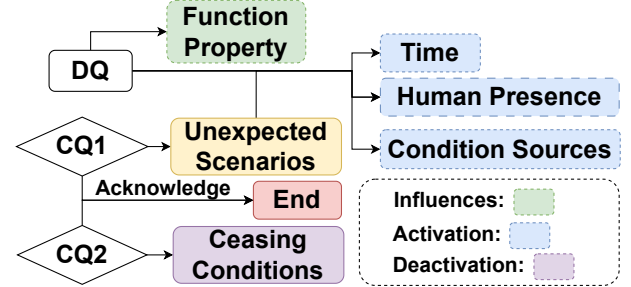


Fig. 4: Preference context construction workflow: *Examiner* analyzes *Users'* answers to infer the activation conditions, deactivation conditions, and influence of apps.

Listing 1: Preference Context Structure

```
Preference Context = {
  "Activation": {
    "Property Conditioning": [Src1, Src2, ...],
    "Unwanted Conditioning": [Src1, Src2, ...],
    "Temporal Conditioning":
      [Time1, Time2, ...],
    "Presence Conditioning":
      [Cond1, Cond2, ...],
  },
  "Functioning": {
    "Main": [Prop1, Prop2, ...],
    "Side-Effects": [Prop1, Prop2, ...]
  },
  "Deactivation": {
    "Sustaining": True/False,
    "Deactivation Conditioning":
      [Cond1, Cond2, ...]
  }
}
```

1) *DQ Reasoning*: The CRA first directly analyzes the design rationales of each app to obtain a fundamental understanding of the Activation process and its conceptual/environmental influences. Specifically, the CRA uses two agents, an *Examiner* and a *User*, for the enumeration. The *Examiner* queries the *User* to describe the scenarios to which the app could be applied. By comparing the description with the original app configurations, the *Examiner* identifies the gap between expectations and actual designs, pinpointing potentially missing conditions—such as temporal conditions or human presence conditions—that can be added without degrading system utility. Additionally, the *Examiner* summarizes the conceptual/environmental influences of the app. Nonetheless, considering that the *User* may fail to disclose real users' subconscious intentions, the CRA further employs counterfactual reasoning to complement existing conditions and the logic of app reversion behaviors.

2) Counterfactual Reasoning on Unintended Interactions:

The fundamental reasons for conflicts are the lack of proper constraints on app executions; i.e., apps can be triggered and can quit execution in more situations than expected. Therefore, the best way to disclose such problems is to directly exhibit counterfactual scenarios. To this end, the CRA uses the *Challenger* to generate counterfactual scenarios of the app to challenge the *User*. The generation process mainly follows three steps: (i) Imagine a scenario where all triggers/conditions could be satisfied; (ii) Substitute the triggering/conditioning sources or add new triggering/conditioning sources so that unsafe/energy-inefficient/insecure/meaningless events happen. Based on the responses from the *User*, the *Examiner* confirms missing constraints that could be included in the app to avoid unintended interactions.

3) *Counterfactual Reasoning on Deactivation*: Current smart home app configurations only support setting a device to a state but do not concern its subsequent behaviors. Yet, previous studies demonstrate that users have expectations regarding rules to “automatically change their action state when appropriate” [26]. As a result, it is important to reason about users’ expectations regarding when to deactivate their apps. Accordingly, the *Challenger* generates several scenarios where the state of devices might be changed (manually or automatically) and queries the *User* about its considerations. The *Examiner* then extracts the *User*’s opinions on the deactivation examples and converts them into a series of requirements (conditions). Depending on the type of requirement, apps are separated into “persistent” apps (action states are expected to be kept for a while) and “instant” apps (action states can be changed instantly).

After the preference enumeration step, the CRA obtains a comprehensive understanding of users’ design rationales and preferences for each app, i.e., the preference contexts. Similarly, PaCR allows users to review and adjust the generated preference contexts for each app to reflect their intentions and preferences.

E. Conflict Handling

In the final CH step, the CRA identifies potential conflicts in the current smart home configurations and proposes resolutions accordingly, as shown in Fig. 5. As defined earlier, the CRA focuses on three types of conflict: PIC, ISC, and DCC, where DCC can be further classified into app-app conflicts (DCC_{app}) and app-user conflicts (DCC_{manual}).

1) *Conflict Handling Principles*: The CRA follows a safety/security-first principle when handling conflicts. Specifically, when conflicts involve safety/security concerns, the CRA prioritizes resolving these issues first to ensure the well-being of users and the security of their smart home environments. Additionally, the CRA will not add new constraints to such apps to maintain the generality of safety/security protocols. In contrast, for conflicts that do not involve safety/security concerns, the CRA aims to minimize utility degradation while addressing the issues. This means that the CRA will seek solutions that preserve as much of the original functionality and user preferences as possible (using as few additional

conditions as possible), ensuring that the smart home system remains efficient and user-friendly even after conflict resolution. Since most conflicts are addressed via condition appending or action creation, the CRA needs to ensure that all newly generated entities follow the same syntactic format as existing apps. Fortunately, with the development of LLM backbones, models have demonstrated promising performance in following structural output formats. Therefore, for conflict resolution, the CRA specifies the output structure of trigger-action-programs via few-shot JSON examples.

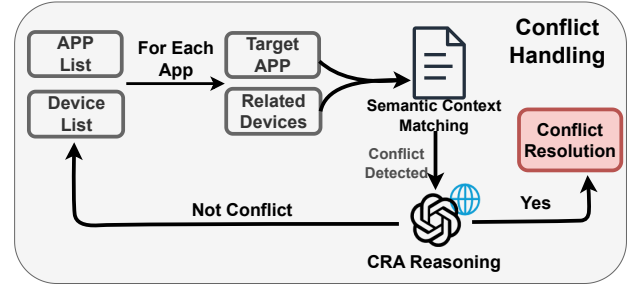


Fig. 5: Details of Conflict Handler. Semantic correlations are systematically examined to discover potential conflicts. CRA further reasons for such conflicts and applies resolution app modifications.

2) *Detect and Handle PIC*: PIC occurs when app actions or devices generate mutual interference, which may further lead to energy waste and unsatisfying living conditions. PaCR leverages **functioning properties** ($PROP_{function}$) of preference contexts to analyze whether an app causes countering effects on the functioning properties of other devices/apps. PaCR obtains the list of devices in the same scene as the app and applies semantic relationship verification to their correlations over the app’s major environmental properties. Upon detection of any potential interferences, PaCR feeds the device and the app into the CRA for conflict reasoning and instructs the CRA to directly modify the app to avoid such conflicts.

Listing 2: PIC Resolution Prompt

You are a smart home expert who identifies unwanted interferences between devices and triggers action programs. You’ll be given a tap and a list of devices that may have countering effects on the current app. Your job is to analyze whether such unexpected interference exists. If so, return true and add new conditions in the ‘triggers’ to constrain the execution. Your return must stick to the original trigger action program JSON format like the following:

```
```json
{
 "result": true/false,
 "app": {
 // Updated tap if interference exists (true); Otherwise leave this empty.
 }
}
```

```
}
}
...
```

3) *Detect and Handle DCC*: *DCC* occurs when two apps or an app and manual operations compete controls over the same device. Therefore, the most intuitive idea to resolve such conflicts is to prioritize different operations. By ranking various apps and operations, higher-order operations are allowed to maintain the device states, and lower-order operations are prohibited, and vice versa. To achieve this, CRA refers to the **deactivation contexts** to determine the priorities among apps and provides different resolution options for *DCC<sub>app</sub>* and *DCC<sub>manual</sub>*.

We first introduce how CRA addresses *DCC<sub>app</sub>*. *DCC<sub>app</sub>* occurs when two apps experience racing controls over devices. Between the two apps, CRA prioritizes persistent apps (with deactivation requirements) before instant apps (without deactivation requirements). This is because, by design, deactivation requirements reflect *Users'* expectations on the sustainability of apps. On the one hand, instant apps are not required to be maintained indefinitely, i.e., any state changing applied to an instant app will not violate users' expectations. On the other hand, persistent apps need to be held until certain conditions are satisfied; i.e., reverse operations should only be applied to the app after the deactivation conditions are satisfied. As a result, given two apps, an instant app should always be conditioned by the deactivation conditions of the persistent app it relates to. Yet when two persistent apps conflict, it becomes necessary for the CRA to determine which app has higher priority. CRA may generate two types of handling options: 1. higher priority app always directly interrupts activated apps; 2. higher priority app temporarily interrupts activated apps, but recovers the states after finishing its tasks.

#### Listing 3: *DCC<sub>app</sub>* Resolution Prompt

```
You are a smart home expert tasked with
identifying unwanted interferences and
manual operations, and triggering action
programs. After a tap is triggered, other
trigger action programs may alter the
state of actions. The user also provides
specified holding conditions for certain
taps. Your job is to analyze whether the
two taps will unexpectedly alter each
other's state (before their corresponding
conditions are satisfied). If so, return
true and create appropriate conditions
using deactivation requirements. Your
return must strictly follow the original
JSON format for trigger action program
conditions, as shown below:
```json
{
  "result": true/false,
  "rules": [
    // Leave empty if false
    {
      // updated rule 1
    },
    {
      //updated rule 2
    }
  ]
}
```

```
]
}
...
...
...

```

DCC_{manual} occurs when users' manual operations change the states, while persistent apps should maintain their action afterwards. Consequently, *DCC_{manual}* is resolved with a post-handling rationale, i.e., CRA creates new recovery apps that restore the functions of persistent apps following manual intervention. Additionally, since it is essential to ensure that newly created recovery apps do not introduce new instances of *DCC_{app}* on other persistent apps, such newly generated recovery apps are regulated by the deactivation requirements of other higher priority persistent apps.

Listing 4: *DCC_{manual}* Resolution Prompt

```
You are a smart home expert tasked with
identifying unwanted interferences and
manual operations, and triggering action
programs. After a tap is triggered, users
or other physical world events may change
the state of actions within the tap. Your
job is to analyze whether the user likely
intends to hold the tap action
temporarily (i.e., whether it is
acceptable to persistently change the
states of action devices while the action
is active). If not, return true (the tap
is expected to sustain) and create
appropriate external conditions that
convey the expectation: "the action
device is expected to change once the
condition is satisfied." A list of
accessible devices will also be provided
for use in the conditions; conditions may
also be specific time ranges. Your return
must strictly follow the original JSON
format for trigger action program
conditions, as shown below:
```

```
```json
{
 "result": true/false, // "true" if the
 tap is expected to sustain(true);
 Otherwise false.
 "ruleWithNewCond": {
 "ruleId": ruleId,
 "rule": [
 //rule with new conditions in the
 same format
]
 }
}
...

```

4) *Detect and Handle ISC*: ISC occurs when smart home devices unexpectedly enable/disable the trigger/conditions of apps. To detect such unexpected interactions, CRA leverages the collected **activation contexts** to determine whether the interactions between devices and app trigger/conditions are appropriate. Specifically, CRA references the activation contexts to identify unwanted conditioning devices and block their interactions with new conditions. In addition, CRA appends

new temporal and human presence conditions to the apps to further regulate their execution to reduce the possibilities of interactions.

#### Listing 5: ISC Resolution Prompt

You are a smart home expert tasked with identifying unwanted interferences between devices and triggering action programs. You will be provided with a tap, a device that may affect the triggers or conditions of the current trigger action program, and a list of user-expected interaction sources. Your job is to analyze whether such interactions actually exist for the specified property, and whether they are unexpected. If so, return true and add new conditions in the 'triggers' field to restrict execution. Your return must strictly follow the original JSON format of the trigger action program, as shown below:

```
```json
{
  "result": true/false, // "true" if
    interactions exist and are
    unexpected; Otherwise false.
  "rule": {
    // Updated rule if interactions exist
    and are unexpected; Otherwise,
    leave this empty.
  }
}
```
```

#### F. Prompts Refinement & Implementations

Prompt engineering builds the foundation of our agentic conflict resolution workflow. However, creating effective prompts for preference elicitation is challenging and time-consuming, and poorly designed prompts lead to ambiguous or irrelevant analysis, undermining the effectiveness of the entire conflict resolution process. To overcome the challenge, CR leverages a meta-prompting model<sup>2</sup> (i.e., the meta-model that creates prompts for all agents) to iteratively generate and refine prompts. The general format of meta-prompt is as follows: *You are a smart home expert as well as a prompt designer who creates prompts for language models to handle various tasks regarding smart home environments. [Detailed descriptions of the target problem and expected output.<sup>3</sup>] Please draft a candidate prompt that can help achieve the goal and help me evaluate their quality based on the follow-up answers to such prompts.*

For each problem-solving (P-S) model (*Examiner*, *User*, etc.), we feed the corresponding candidate prompt and tasks to generate responses, and use the meta-prompting (M-P) model to evaluate the quality of the dialogues and enhance prompts accordingly. Such a process iterates several rounds until the

M-P model believes that the output quality of the P-S model is satisfactory. Nonetheless, due to the lack of ground-truth references, the M-P model exhibits difficulties in objectively evaluating the output quality of P-S models, especially for responses from the *User* sub-agent.

To address the problem, we carried out a pilot survey within the lab and discussed with all participants to dig out their subconscious intentions on app design and interpretations of app conflicts. Our survey consisted of 7 conflict-interpretation tasks and 3 app-creation tasks to explore users' preferences and expectations for smart home interactions across various contexts. During the survey, participants were queried with the same DQ, CQ1, CQ2, and conflict interpretation questions. We manually examined their answers, corrected the ambiguous ones via a discussion session, and formalized their answers into the expected agent answer format. Consequently, these answers from real participants serve as the ground-truth that M-P models can refer to to assess the quality of answers from P-S models.

### V. EXPERIMENTS & EVALUATION

In this section, we evaluate the efficacy and utility of PaCR for conflict detection and resolution. To assess our system on a broader scale of app sets and real user feedback, we conducted a quantitative evaluation on existing datasets [8] and a qualitative user study within simulated environments. The overall evaluation workflow comprises three main phases: **1.** Quantitative evaluation at scale; **2.** Closed-world quantitative comparisons; and **3.** Real user evaluations of closed-world performance.

#### A. Quantitative Evaluation Design

We use the app sets collected from real users [8] for our quantitative evaluation at scale. The dataset provides 126 app sets, among which 44 app sets were eliminated from the evaluation as they contained fewer than 10 apps in total. For the remaining 82 app sets, we standardize their trigger-action programs into our standard app format as follows:

- **Description:** Description of the app.
- **Trigger:** Original app *triggers*. If the *triggers* section contains multiple trigger entities, we select the first one as the main trigger.
- **Condition:** All entities in the *triggers* excluding the **Trigger**. Left empty if the *triggers* section contains only one event.
- **Action:** All actions in the *actions* section.

These app sets cover three common scenarios in smart homes: the living room, the study room, and the bedroom. Each app set contains 10 to 30 apps (one outlier user created 75 apps), with an average of 27.3 apps per set. Each app contains an average of 1.23 triggers/conditions and 1 action. For quantitative evaluation, we compare PaCR with TapChecker [8] on the same dataset. Due to the lack of ground-truth conflicts, the comparison focuses on the quantity of each type of conflict detected by different methods. Moreover, we use TapChecker as the baseline for evaluating the correctness of app-app conflicts due to its formal verification capabilities.

<sup>2</sup>We adopted GPT-5 as the M-P model for its strong reasoning capability.

<sup>3</sup>EXAMPLE: "Extract potential app functionalities and their environmental impacts; The output needs to be structural: FORMAT:"



Apart from the quantitative evaluation, we noticed from the aforementioned distributions that apps in this dataset mostly contain a limited number of triggers, conditions, and actions. Consequently, conflicts easily arise among apps and devices. Additionally, such simple app designs suggest that most app sets fail to cover complex conflict scenarios. Therefore, we further conducted a closed-world evaluation incorporating real user feedback.

### B. Closed-World Evaluation Design

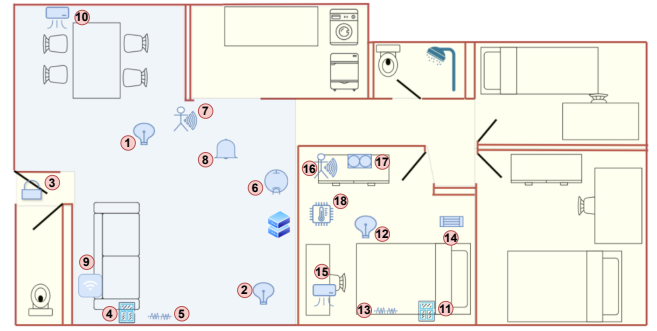
One major limitation of the above large-scale evaluation is the lack of ground-truth conflicts. There is no reference for how many conflicts exist within the app sets, and it is extremely labor-intensive to label these conflicts manually. As a result, such an evaluation cannot measure the accuracy of conflict detection and the effectiveness of handling outcomes.

To obtain a fine-grained analysis of PaCR, we crafted a closed-world smart home configuration with ground-truth conflicts (Table II) as a controlled experiment. Specifically, these apps encompass all types of potentially conflicting interactions (Table IV), created following three principles: 1. They are typical customized apps found in existing works [5], [11], [8], [40] (e.g., APP #1, #2, #3, #4, #7, #11); 2. The automation deliberately includes a type of conflict in our template definitions (e.g., APP #5, #6, #8); and 3. The closed-world configuration includes many smart devices that natively serve various purposes, such as windows, A/C, and curtains, to increase the complexity of the analysis.

After crafting the smart home configuration, we applied PaCR to handle all conflicts and obtained the resolution results. Since such systems are designed to assist users in conflict resolution, we further conducted a user study where participants compared the same handling results (including the reasoning process regarding conflicts) to the original app sets and rated their satisfaction level with PaCR. To this end, we built a survey website to better demonstrate such results to participants and instruct them to rate each result accordingly, as shown in Fig. 6. During the evaluation, participants were first introduced to the concepts of smart home automation, conflicts, and the set of predefined apps. After familiarizing themselves with the system, we demonstrated all handling results to participants (Fig 7) and asked them to rate the handling results on a scale of 1-5, where **5** indicates “very satisfying” and **1** indicates “unsatisfying”. Moreover, participants were asked to provide feedback on unsatisfying results to help us identify potential areas for improvement.

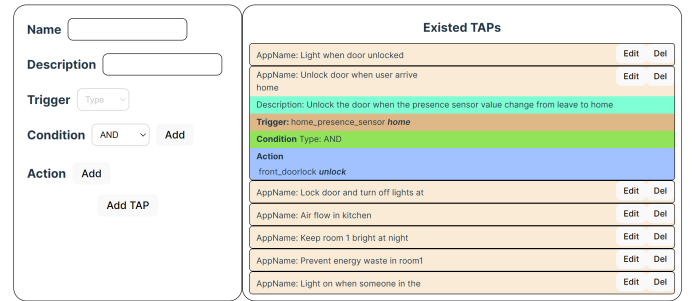
### C. Implementation

In our experiments, we implemented PaCR using GPT-5-Mini as the agent backbone due to its efficiency and relatively low cost. The meta-prompting model was implemented with GPT-5 to leverage its stronger deep reasoning capabilities. The interaction workflow is orchestrated using Python-OpenAI APIs. We developed the website with Vue3 and Flask to simulate smart home configurations and collect user feedback.



Current area: Living Room

(a) Screen shot of the simulated smart home floorplan.



(b) Screenshot of simulated app configuration.

Fig. 6: Simulated smart home configuration.

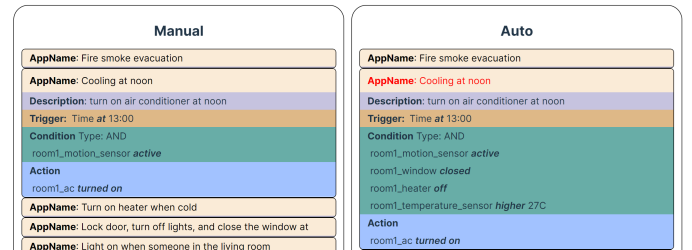


Fig. 7: Screen shot of the comparison component for micro scope ratings.

### D. Quantitative Evaluation Results

The quantitative evaluation results are summarized in Table I. Overall, PaCR detected 5283 conflicts across 82 app sets, with an average of 64.4 conflicts per app set. Among these, 216 are  $DCC_{app}$  conflicts, 1839 are  $DCC_{manual}$  conflicts, 1591 are ISC conflicts, and 1637 are PIC conflicts. In contrast, TapChecker identified only 536 potential conflicts in total. Such a large gap is attributed to TapChecker’s focus on app-app conflict detection. By design, TapChecker only examines interactions between apps and overlooks those caused by manual device operations, leading to a significant underestimation of the overall conflict landscape. For instance, users mostly create apps for air conditioner control, while some devices (e.g., windows, doors) are less involved in their app designs. However, the states of these devices still have a significant impact on the functionality of temperature control apps. There-

fore, it is crucial to consider device-app interactions in conflict detection to ensure a comprehensive understanding of potential issues in smart home environments, i.e., examining PIC, ISC, and  $DCC_m$  conflicts.

Furthermore, TapChecker was used to assess the completeness of PaCR's conflict-resolution outcomes for app-app conflicts. When applied to the resolved app sets, **no residual conflicts were identified**, demonstrating that PaCR effectively eliminates inter-app conflicts. Regarding device-app conflicts, manual validation was conducted on the first 10 app sets and their associated conflict reasoning frameworks as case studies. Most device-app conflicts were appropriately addressed with logically rigorous reasoning, but we also observed instances of overly conservative resolution strategies that led to utility degradation. For example, PaCR detected a conflict between the app "open curtain when human motion is detected" and the TV device, with the rationale that "opening the curtain could lead to glare from natural light, which may impair the TV viewing experience." While incorporating additional constraints effectively mitigates the purported interference, it also limits the app's applicability under certain contextual conditions. We quantitatively evaluate such situations with precision rates, where false positive results refer to those that unnecessarily degrade app functionality without genuine conflict risks. The precision rates for  $DCC_{app}$ ,  $DCC_{manual}$ , ISC, and PIC were 98%, 99%, 92%, and 97%, respectively.  $DCC_{manual}$  and  $DCC_{app}$  yielded the highest precision rates, as their conflict scenarios are more straightforward and easier to validate, i.e., identifying whether racing condition over the same device exists. ISC conflicts had the lowest precision rate due to the complexity of inferring user preferences, which may lead to occasional misinterpretations. To further investigate false positive issues, we gathered empirical user feedback on closed-world resolution outcomes, as elaborated in the subsequent sections.

### E. Quantitative Closed-World Evaluation

In the closed-world evaluation study, we thoroughly assess the effectiveness of PaCR's conflict handling results and pinpoint potential limitations. This begins with comparisons against three SOTA approaches (IoTMon [5], CP-IoT [11], and AutoIoT [40]) over the same app set. The evaluation focuses on two key aspects: threat detection capability and the appropriateness of app modifications. The detailed handling results are displayed in Table III.

As shown in Table IV, PaCR detects more threats than all SOTA approaches. This is primarily because prior works define and focus on conflicts between apps, rendering them less comprehensive. In addition, PaCR provides appropriate app modifications due to its preference-alignment feature, as demonstrated in Table III. Previous works either use a generic approach (IoTMon) or a consistent handling strategy (CP-IoT) to resolve threats, which significantly degrades system utility. For instance, CP-IoT detects all six  $DCC_{app}$  conflicts but only resolves two of them appropriately. Its strategy prioritizes apps based on features: "User Activity > Physical-Related > Physical-Unrelated > Mode-related > Time-related," because

| UID          | App Num | $DCC_a$    | $DCC_m$     | ISC         | PIC         | Total       |
|--------------|---------|------------|-------------|-------------|-------------|-------------|
| 5            | 30      | 19         | 28          | 22          | 29          | 98          |
| 13           | 31      | 9          | 28          | 31          | 24          | 92          |
| 14           | 20      | 1          | 20          | 18          | 19          | 58          |
| 15           | 20      | 3          | 18          | 16          | 17          | 54          |
| 16           | 21      | 5          | 18          | 18          | 20          | 61          |
| 17           | 10      | 0          | 10          | 9           | 10          | 29          |
| 18           | 26      | 0          | 25          | 20          | 22          | 67          |
| 19           | 21      | 0          | 20          | 21          | 17          | 58          |
| 20           | 20      | 4          | 19          | 15          | 16          | 54          |
| 21           | 20      | 1          | 19          | 14          | 14          | 48          |
| 23           | 20      | 5          | 20          | 19          | 19          | 63          |
| 24           | 24      | 1          | 24          | 23          | 22          | 70          |
| 25           | 25      | 0          | 24          | 21          | 24          | 69          |
| 26           | 28      | 10         | 28          | 26          | 22          | 86          |
| 31           | 22      | 2          | 22          | 16          | 18          | 58          |
| 32           | 28      | 4          | 25          | 22          | 25          | 76          |
| 33           | 21      | 0          | 21          | 17          | 16          | 54          |
| 36           | 22      | 0          | 21          | 17          | 20          | 58          |
| 37           | 25      | 2          | 25          | 22          | 20          | 69          |
| 39           | 23      | 1          | 21          | 18          | 22          | 62          |
| 40           | 28      | 2          | 27          | 23          | 24          | 76          |
| 41           | 22      | 2          | 21          | 17          | 20          | 60          |
| 43           | 22      | 0          | 22          | 20          | 19          | 61          |
| 44           | 24      | 2          | 21          | 21          | 21          | 65          |
| 45           | 25      | 0          | 25          | 19          | 22          | 66          |
| 46           | 22      | 4          | 20          | 15          | 15          | 54          |
| 47           | 26      | 3          | 24          | 20          | 23          | 70          |
| 48           | 30      | 4          | 23          | 23          | 21          | 71          |
| 49           | 25      | 6          | 25          | 22          | 21          | 74          |
| 50           | 22      | 0          | 21          | 17          | 14          | 52          |
| 51           | 26      | 6          | 24          | 21          | 22          | 73          |
| 52           | 23      | 3          | 19          | 19          | 19          | 60          |
| 55           | 22      | 4          | 20          | 14          | 15          | 53          |
| 57           | 30      | 9          | 29          | 26          | 26          | 90          |
| 58           | 25      | 0          | 22          | 23          | 19          | 64          |
| 60           | 26      | 2          | 23          | 22          | 17          | 64          |
| 63           | 23      | 0          | 23          | 18          | 18          | 59          |
| 64           | 22      | 2          | 22          | 18          | 20          | 62          |
| 66           | 22      | 0          | 21          | 17          | 20          | 58          |
| 67           | 15      | 0          | 15          | 14          | 14          | 43          |
| 68           | 22      | 0          | 22          | 17          | 15          | 54          |
| 69           | 22      | 3          | 21          | 15          | 19          | 58          |
| 70           | 25      | 0          | 25          | 22          | 20          | 67          |
| 71           | 18      | 2          | 16          | 15          | 17          | 50          |
| 73           | 18      | 4          | 18          | 16          | 17          | 55          |
| 75           | 20      | 2          | 20          | 17          | 19          | 58          |
| 77           | 18      | 3          | 18          | 16          | 18          | 55          |
| 81           | 30      | 3          | 30          | 26          | 22          | 81          |
| 82           | 26      | 5          | 23          | 20          | 23          | 71          |
| 83           | 75      | 18         | 71          | 62          | 61          | 212         |
| 84           | 30      | 8          | 30          | 24          | 26          | 88          |
| 85           | 30      | 2          | 27          | 26          | 21          | 76          |
| 89           | 25      | 2          | 24          | 19          | 23          | 68          |
| 94           | 20      | 0          | 19          | 18          | 16          | 53          |
| 100          | 25      | 2          | 21          | 18          | 24          | 65          |
| 103          | 20      | 2          | 18          | 14          | 18          | 52          |
| 104          | 15      | 2          | 14          | 14          | 13          | 43          |
| 109          | 18      | 1          | 18          | 15          | 16          | 50          |
| 110          | 22      | 4          | 22          | 20          | 20          | 66          |
| 111          | 31      | 2          | 29          | 23          | 24          | 78          |
| 112          | 18      | 1          | 17          | 12          | 17          | 47          |
| 113          | 19      | 0          | 18          | 16          | 15          | 49          |
| 114          | 24      | 1          | 22          | 16          | 20          | 59          |
| 115          | 31      | 4          | 29          | 16          | 22          | 71          |
| 116          | 20      | 3          | 19          | 18          | 17          | 57          |
| 117          | 22      | 1          | 19          | 19          | 21          | 60          |
| 119          | 30      | 1          | 29          | 21          | 17          | 68          |
| 121          | 18      | 5          | 16          | 18          | 13          | 52          |
| 122          | 25      | 3          | 22          | 19          | 21          | 65          |
| 123          | 24      | 2          | 24          | 15          | 21          | 62          |
| 125          | 19      | 0          | 18          | 17          | 15          | 50          |
| 126          | 22      | 1          | 18          | 20          | 16          | 55          |
| 128          | 19      | 0          | 18          | 19          | 18          | 55          |
| 130          | 30      | 2          | 29          | 24          | 24          | 79          |
| 131          | 18      | 0          | 17          | 13          | 17          | 47          |
| 132          | 30      | 4          | 30          | 23          | 26          | 83          |
| 134          | 18      | 0          | 14          | 15          | 17          | 46          |
| 135          | 24      | 0          | 22          | 24          | 24          | 70          |
| 136          | 28      | 1          | 28          | 21          | 25          | 75          |
| 137          | 21      | 0          | 21          | 18          | 18          | 57          |
| 138          | 20      | 2          | 19          | 17          | 17          | 55          |
| 140          | 21      | 4          | 21          | 19          | 18          | 62          |
| <b>Total</b> |         | <b>216</b> | <b>1839</b> | <b>1591</b> | <b>1637</b> | <b>5283</b> |

TABLE I: Quantities of different types of conflicts detected in the 82 app sets.

user/physical-related apps are more relevant to safety and security issues, while mode/time-related apps tend to be routine and more regulated. Unfortunately, this approach severely degrades system safety and security levels in customized app set scenarios. Regarding *APP1* and *APP2*, CP-IoT allows *APP2* to leave the door unlocked overnight when the user

| App ID | Automation Description                                                                                     |
|--------|------------------------------------------------------------------------------------------------------------|
| 1      | At 11:00 p.m., lock the front door (3), turn off living room lights (1) and (2), and close the window (4). |
| 2      | When the presence sensor (9) detects the user getting back, unlock the front door for convenience.         |
| 3      | When motion (7) detected in the living room, turn on light (1).                                            |
| 4      | When smoke (8) detected, open window (4).                                                                  |
| 5      | Everyday at 4:00 p.m., open the window (4) for air circulation.                                            |
| 6      | Everyday at 4:30 p.m., close the window (4) after freshening the air.                                      |
| 7      | If light (12) is on and the time is after 6:00 p.m., close the curtain (13) for privacy.                   |
| 8      | When motion (16) detected, if illuminance (17) not enough, open the curtain (13).                          |
| 9      | Everyday at 1:00 p.m., if motion detected (16), turn on the air conditioner (15).                          |
| 10     | If temperature (18) is lower than 26°C, turn off the air conditioner (15).                                 |
| 11     | If temperature (18) is lower than 16°C, turn on the heater (14).                                           |
| 12     | At the weekend afternoon, turn on robot vacuum (6) to clean the whole apartment.                           |

TABLE II: Customized automation apps for the pilot study and real user evaluation.

| App ID | App-modification Results                                                                                                                                                                            |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1      | At 11:00 p.m., <u>lock the front door</u> .                                                                                                                                                         |
| 2      | When <u>presence sensor</u> detects user getting back, <b>if time is not between 11:00 p.m. and 6:00 a.m.</b> , unlock the <u>front door</u> for convenience.                                       |
| 3      | When <u>motion</u> is detected in the living room, <b>if time is "Night"</b> , and if the <u>window, curtain, and vacuum</u> are not working, turn on the <u>light</u> .                            |
| 4      | When <u>smoke</u> is detected, open the <u>window</u> .                                                                                                                                             |
| 5      | At 4:00 p.m., open the window for air circulation.                                                                                                                                                  |
| 6      | Everyday at 4:30 p.m., <b>if smoke not detected</b> , close the <u>window</u> after freshening the air.                                                                                             |
| 7      | When <u>light</u> is on and time is after 6:00 p.m., close the <u>curtain</u> for privacy.                                                                                                          |
| 8      | When <u>motion</u> is detected, <b>if time is "Day"</b> , and if <u>window, curtain, and vacuum</u> are not working, and if <u>illuminance</u> is not enough, open the <u>curtain</u> .             |
| 9      | Everyday at 1:00 p.m., if <u>motion</u> is detected, and if <u>window</u> is not opened, <u>heater</u> is not on, and <b>temperature is higher than 26°C</b> , turn on the <u>air conditioner</u> . |
| 10     | When <u>temperature</u> is lower than 26°C, turn off the <u>air conditioner</u> .                                                                                                                   |
| 11     | When <u>temperature</u> is lower than 16°C, <b>if a user presents</b> , and if the <u>window</u> is not opened and <u>A/C is not on</u> , turn on the <u>heater</u> .                               |
| 12     | At the weekend afternoon, turn on the robot vacuum to clean the whole apartment.                                                                                                                    |
| 13     | At 11:00 p.m., <b>if motion is not detected</b> , turn off the <u>living room lights</u> .                                                                                                          |
| 14     | At 11:00 p.m., <b>if smoke is not detected</b> , close the <u>window</u> .                                                                                                                          |
| 15     | <i>When front door unlocked, if time is between 11:00 p.m. and 6:00 a.m., lock the door after 30 seconds.</i>                                                                                       |
| 16     | <i>When the window is closed, if smoke is detected, open the window after 10 seconds.</i>                                                                                                           |
| 17     | <i>When the curtain is opened, if time is between 6:00 p.m. and 5:00 a.m., close the curtain after 10 minutes.</i>                                                                                  |

TABLE III: Handling results of PaCR. Gray color denotes original trigger/conditions/actions; Brown color indicates the results of the automatic function separation. Cyan color indicates the "Time" and "Presence" conditions added; New conditions related to the "State Holding Requirement" are marked with Red color; Other new trigger/conditions/actions added during threat handling are marked with Blue color; Newly created apps are displayed with *Italic* texts. All devices (entities) are underlined for clarity of viewing.

comes back home after 11:00 p.m. As for APP1 and APP3 regarding light control, CP-IoT blocks the entire APP1 due to its time-related feature, leaving the door unlocked and the window open overnight. It turns out that customized smart home apps can have different security/safety implications under different contexts and for different individuals compared to official ones. AutoIoT only detects potential conflicts between two apps and thus ignores the problems between manual operations and the apps. In addition, since AutoIoT overly relies on LLMs for app optimization without sufficient constraints, the handling options are sometimes too random to fulfill users' requirements, even though conflicts are resolved.

#### F. Qualitative Closed-World Evaluation

To further obtain statistically meaningful evaluations of the handling results, we recruited participants from Prolific [60] to quantitatively evaluate all handling outcomes. The recruited participants were required to have experience with smart homes, and no other screening criteria were applied. In the end, we recruited 20 participants. The demographics of the 20 participants are displayed in Table V.

Participants' satisfaction levels regarding the handling results are demonstrated in Table VI. Overall, the majority of participants gave high ratings for most modification suggestions, with a median rating of 4.4 out of 5. To further investigate the root causes of dissatisfaction with certain resolutions, we identified the key issues among the lower-rated cases.

**Over-reasoning of Potential Interactions.** LLMs' reasoning results remain not fully reliable [58], [61], [62]. As mentioned in Section V-D, overinterpretation of conflicting interactions indirectly contributes to unsatisfactory results. For instance, most users agree that a potential interaction source conflict exists between APP #8 and the robot vacuum: the moving vacuum could trigger the motion sensor originally designed to detect human presence. Interestingly, when applying similar reasoning to APP #9, users' consensus no longer exists, despite both conflicts stemming from the same unexpected interaction between the motion sensor and robot vacuum.

We propose a possible explanation for such a phenomenon: the probability of occurrence significantly influences conflict interpretation. When illuminance is insufficient, the interaction between APP #8 and the robot vacuum consistently triggers

| Threat Type           | Conflict APP ID & Object | Description                                                                  | Detection / Handling |             |              |            |
|-----------------------|--------------------------|------------------------------------------------------------------------------|----------------------|-------------|--------------|------------|
|                       |                          |                                                                              | IoTMon [5]           | CP-IoT [11] | AutoIoT [40] | PaCR(ours) |
| PIC                   | 9 & Window               | Window (11) is opened before 9 executes                                      | × / ×                | × / ×       | × / ×        | ✓ / ✓      |
| PIC                   | 11 & Window              | window (11) is opened before 11 executes                                     | × / ×                | × / ×       | × / ×        | ✓ / ✓      |
| DCC <sub>app</sub>    | 1 & 2                    | Users arrive home after 11:00 p.m. and APP2 unlocks the front door (3)       | × / ×                | ✓ / ×       | ✓ / ✓        | ✓ / ✓      |
| DCC <sub>app</sub>    | 1 & 3                    | Light (1) turned on at 11:00 p.m. when someone is still in the living room   | × / ×                | ✓ / ×       | ✓ / ✓        | ✓ / ✓      |
| DCC <sub>app</sub>    | 4 & 1                    | Smoke detected before 11:00 p.m., window (4) gets closed                     | × / ×                | ✓ / ×       | ✓ / ✓        | ✓ / ✓      |
| DCC <sub>app</sub>    | 4 & 6                    | Smoke detected before 4:30 p.m., window (4) gets closed                      | × / ×                | ✓ / ✓       | ✓ / ✓        | ✓ / ✓      |
| DCC <sub>app</sub>    | 9 & 10                   | A/C (15) turned on when the temperature is lower than 26°C                   | × / ×                | ✓ / ✓       | ✓ / ✓        | ✓ / ✓      |
| DCC <sub>app</sub>    | 7 & 8                    | Curtain (13) is opened by APP8 after 6:00 p.m.                               | × / ×                | ✓ / ×       | ✓ / ✓        | ✓ / ✓      |
| DCC <sub>manual</sub> | 1 & Manual unlock        | Users arrive home and manually unlock the door (3) after 11:00 p.m.          | × / ×                | × / ×       | × / ×        | ✓ / ✓      |
| DCC <sub>manual</sub> | 4 & Accidental close     | Any accidental physical force that closes the window (4)                     | × / ×                | × / ×       | × / ×        | ✓ / ✓      |
| DCC <sub>manual</sub> | 7 & Temporary open       | Users temporarily open the curtain (13) after 6:00 p.m.                      | × / ×                | × / ×       | × / ×        | ✓ / ✓      |
| ISC/PIC               | 11 & A/C                 | A/C (15) may lower the temperature                                           | ✓ / ○                | ✓ / ✓       | × / ×        | ✓ / ✓      |
| ISC/PIC               | 9 & Heater               | Heater (14) may increase the temperature                                     | ✓ / ○                | ✓ / ✓       | × / ×        | ✓ / ✓      |
| ISC                   | 3 & Motion               | The motion of (4), (5), and (6) may trigger living room motion sensor        | ✓ / ○                | ✓ / ✓       | × / ×        | ✓ / ✓      |
| ISC                   | 8 & Motion               | The motion of (6), (11), and (13) may trigger motion sensor                  | ✓ / ○                | ✓ / ✓       | × / ×        | ✓ / ✓      |
| ISC(FP)               | 4 & Heater               | False positive interactions detected between heater (13) and smoke alarm (8) | ✓ / ✓                | N/A         | ✓ / ✓        | N/A        |

TABLE IV: Closed-world comparisons between the state-of-the-art methods and PaCR. All manually identified threats serve as the ground truth, and systems are expected to detect and handle such threats appropriately. The threats exist between an app (identified by its ID) and an object (e.g., an app, a device, or a manual operation). ✓ denotes “a system has identified/handled this threat”. × denotes “a system does not detect the threat or provide corresponding handling.” ○ means the handling processes are mentioned but not introduced in detail. N/A means the system neither detects nor attempts to resolve the situation.

| Profile | Age   |       |       |       |     | Education |    |    |       | Device Owned |      |     |
|---------|-------|-------|-------|-------|-----|-----------|----|----|-------|--------------|------|-----|
|         | 16-24 | 25-34 | 35-44 | 45-54 | 55+ | HS        | Bs | Ms | Ph.D. | <5           | 5-10 | >10 |
| No.     | 8     | 3     | 5     | 2     | 0   | 1         | 16 | 3  | 0     | 10           | 6    | 4   |

TABLE V: Demographics of the 20 participants. HS stands for “High School.”

| Handling results |         |       |     |        |
|------------------|---------|-------|-----|--------|
| Ratings          | Average | Max   | Min | Median |
|                  | 4.2/5   | 5.0/5 | 2/5 | 4.4/5  |

TABLE VI: Ratings for handling results. High rating means higher satisfaction.

app execution. However, APP #9’s execution conditions are more stringent, requiring the robot vacuum to operate and be detected by the motion sensor precisely at 1:00 p.m.—a far less probable scenario. Consequently, fixing extremely rare conflicts degrades app generality, even if the reasoning seems correct. This suggests that incorporating conflict probability assessment could enhance user-friendly conflict handling in the future.

### G. Evaluating PaCR Sub-Components

1) *Semantic Analysis Performance*: In PaCR, we use LLMs to analyze semantic relationships between devices and environments, and leverage these semantic relationships for

potential conflict detection. The prompt template we used in the experiment is as follows:

**Prompt:** Please describe the function of the device, how it influences the environments, and what’s the effect after activating/deactivating it? Please think step by step. The following are a few examples: [EXAMPLES]. Your response should be in JSON format including the following fields: deviceName, function, states, and influences. Now, please analyze the device: [DEVICE NAME].

In the end, SRA reaches **97%** correctness, while Word2Vec models only achieve **78%** accuracy. During implementation, we noticed that some works [8] manually crafted a knowledge base for such semantic relationships, which lacks scalability and is error-prone. According to our manual examination, their entity-device reference dictionary misses **30%** of correlations in our evaluation set, leading to an incomplete analysis of property interference conflicts.

2) *Preference Context*: In PaCR, preference contexts build the foundation of preference-aligned conflict resolution. To demonstrate its performance, we compared the results against the baseline where only devices, apps, and conflict defini-

tions are fed into GPT-5-mini for reasoning, as shown in Table VII. For conflicts/cascading effects between apps, GPT-5-mini detects 70% of the conflicts. As for other potential conflict types between devices and apps, the baseline generates 4 false negative judgments due to the under-reasoning of the semantic influences/relationships. This may be because existing models are not fine-tuned on smart home corpora regarding automation conflicts. As a result, preference contexts provide specific information for LLMs so that models only need to analyze whether the interaction violates the unexpected conditions, significantly lowering the reasoning load.

| Conflict Types | TP | FP | FN |
|----------------|----|----|----|
| App-App        | 7  | 3  | 0  |
| Device-App     | 5  | 1  | 4  |

TABLE VII: Baseline GPT-5-mini reasoning without preference contexts.

#### H. System Time Complexity and Cost

The time complexity of the system is  $O(MNk)$ , where  $M$  is the number of apps (or devices),  $N$  is the number of devices, and  $k$  is the number of properties that correlate two devices or an app and a device.

We also evaluated the actual time cost and overhead of using PaCR to identify potential system bottlenecks. We used the OpenAI GPT-5-mini model as the main agent reasoning backbone. For the preference enumeration phase, each app costs 38 seconds on average. For conflict resolution, each type of conflict examination of a single app costs 15 seconds on average. Therefore, the existing agent-based framework exhibits a certain level of time overhead. While the process is completely automated, we still aim to reduce the time cost for a better user experience in the future. A potential improvement is to separate some reasoning tasks into structured function calling to reduce the load on LLMs.

## VI. DISCUSSION

### A. Adaptation to Various Platforms

PaCR maintains platform adaptability due to its platform-agnostic feature: it only modifies apps to proactively resolve conflicts rather than intervening in device executions at runtime. For the most common mobile-based smart home configuration pages, PaCR follows the approach of [11] to collect smart home setups. This method employs natural language understanding to extract information from app configuration interfaces, enabling accurate cross-platform automation app reconstruction. As PaCR's conflict detection and resolution mechanisms are fundamentally platform-independent, the system can seamlessly adapt to diverse smart home ecosystems using this consistent information acquisition framework.

### B. Scalability, Automation, and Accuracy

In PaCR, many steps rely on LLM reasoning to approximate user preferences, where minor discrepancies may accumulate along steps and finally affect overall system performance. For instance, user profiles are generated based on existing app

configurations, they are not guaranteed to fully capture users' all subconscious intentions and preferences. This is a common trade-off in LLM-systems where models provide systems with strong scalability and automation capabilities, but their accuracy remains black boxed. PaCR can mitigate this issue through Human-in-the-loop feedback, allowing users to review and adjust intermediate outputs, conflicts, and resolutions, thereby refining system results based on real user feedback. Future work could further enhance accuracy by integrating hybrid approaches that combine LLM reasoning with rule-based checks or machine learning models trained on historical conflict resolution data.

### C. LLM Reasoning Limitations and Mitigation Strategies

While PaCR leverages LLMs' semantic reasoning to enhance conflict detection and resolution, the experiments reveal inherent limitations of LLMs that impact system performance, specifically over-reasoning. To mitigate these issues, future iterations of PaCR could integrate two complementary strategies: (i) probability-aware conflict reasoning, which quantifies the likelihood of a detected interaction (e.g., based on historical device usage patterns, time-of-day correlations, or user activity logs) and prioritizes resolution only for conflicts with occurrence probabilities above a user-defined threshold. Recent progress in code generation [63], [64] has proven the potential of dynamically constructing formal verification simulations for suspicious conflicts. Via Monte Carlo sampling [65] from the simulation, PaCR could quantitatively measure the likelihood of a conflict coincidence. (ii) LLM fine-tuning with smart home automation data, which retrains the LLM backbone (e.g., Qwen, Llama) on a curated dataset of smart home conflict cases (including user-labeled "non-conflicts" for rare interactions). This fine-tuning would align LLMs' reasoning with real user perceptions, reducing false positives caused by over-interpretation.

### D. Preference Adaptation via Activity Logs

PaCR generates preference contexts based on the depicted user profile. However, existing user profiles are extracted via LLM-reasoning on the app sets, which cannot provide algorithmic guarantees on the accuracy of user preference enumerations. To enhance authenticity, PaCR could be dynamically adapted by incorporating smart home activity logs that reflect more sequential execution relationships among devices. As a result, such information could be used to validate and refine the initially enumerated preferences. Such information has also been used in prior works [32], [33] for automation generation.

## VII. CONCLUSION

In this paper, we propose PaCR, a preference-aligned agentic conflict resolving framework. Based on concrete reasoning on a given app set, PaCR profiles users' potential living habits and preferences on their smart home environments. Leveraging such preference contexts, PaCR orchestrates agentic workflows to detect and handle potential conflicts in a personalized manner. In addition, we demonstrated that PaCR addresses a broader range of conflicts and exhibits satisfying performance in real user evaluations.



## VIII. ACKNOWLEDGEMENT

This work was supported by the National Natural Science Foundation of China (Grant No: 62372122), the Research Grants Council (Grant No: 15209922), the Innovation and Technology Fund (Grant No: GHP/392/22GD), Hong Kong SAR, China, and PolyU Research Centre for Privacy and Security Technologies in Future Smart Systems.

## REFERENCES

- [1] IFTTT, "Ifittt webpage." [Online]. Available: <https://ifttt.com/>
- [2] H. Assistant, "Home assistant webpage." [Online]. Available: <https://www.home-assistant.io/>
- [3] Samsung, "Smarthings platform," accessed: Oct 2024. [Online]. Available: <https://www.samsung.com/us/smarthings/>
- [4] XiaoMi, "Mihome platform," accessed: Oct 2024. [Online]. Available: <https://www.mi.com/us/smart-home/>
- [5] W. Ding and H. Hu, "On the safety of iot device physical interaction control," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS 2018, Toronto, ON, Canada, October 15-19, 2018*, D. Lie, M. Mannan, M. Backes, and X. Wang, Eds. ACM, 2018, pp. 832–846.
- [6] H. Chi, Q. Zeng, X. Du, and J. Yu, "Cross-app interference threats in smart homes: Categorization, detection and handling," in *50th Annual IEEE/IFIP International Conference on Dependable Systems and Networks, DSN 2020, Valencia, Spain, June 29 - July 2, 2020*. IEEE, 2020, pp. 411–423.
- [7] Z. B. Celik, P. D. McDaniel, and G. Tan, "Soteria: Automated iot safety and security analysis," in *2018 USENIX Annual Technical Conference, USENIX ATC 2018, Boston, MA, USA, July 11-13, 2018*, H. S. Gunawi and B. C. Reed, Eds. USENIX Association, 2018, pp. 147–158.
- [8] L. Chen, C. Wang, C. Chen, C. Huang, X. Chen, and M. Zhang, "Tapchecker: A lightweight smt-based conflict analysis for trigger-action programming," *IEEE Internet of Things Journal*, 2024.
- [9] D. Sun, L. Hu, G. Wu, Y. Xing, J. Hu, and F. Wang, "Threat detection in trigger-action programming rules of smart home with heterogeneous information network model," *IEEE Internet of Things Journal*, 2024.
- [10] H. Chi, Q. Zeng, and X. Du, "Detecting and handling iot interaction threats in multi-platform multi-control-channel smart homes," in *32nd USENIX Security Symposium (USENIX Security 23)*, 2023, pp. 1559–1576.
- [11] H. Lin, C. Li, J. Yang, Z. Wang, L. Fan, and C. Duan, "CP-IoT: A cross-platform monitoring system for smart home," in *31th Annual Network and Distributed System Security Symposium, NDSS 2024, San Diego, California, USA, February 26-March 1, 2024*. The Internet Society, 2024.
- [12] Q. Wang, P. Datta, W. Yang, S. Liu, A. Bates, and C. A. Gunter, "Charting the attack surface of trigger-action iot platforms," in *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, CCS 2019, London, UK, November 11-15, 2019*, L. Cavallaro, J. Kinder, X. Wang, and J. Katz, Eds. ACM, 2019, pp. 1439–1453.
- [13] W. Ding, H. Hu, and L. Cheng, "Iotsafe: Enforcing safety and security policy with real iot physical interaction discovery," in *28th Annual Network and Distributed System Security Symposium, NDSS 2021, virtually, February 21-25, 2021*. The Internet Society, 2021.
- [14] Z. B. Celik, G. Tan, and P. D. McDaniel, "Iotguard: Dynamic enforcement of security and safety policy in commodity iot," in *26th Annual Network and Distributed System Security Symposium, NDSS 2019, San Diego, California, USA, February 24-27, 2019*. The Internet Society, 2019.
- [15] S. Manandhar, K. Moran, K. Kafle, R. Tang, D. Poshyvanyk, and A. Nadkarni, "Towards a natural perspective of smart homes for practical security and safety analyses," in *2020 IEEE Symposium on Security and Privacy, SP 2020, San Francisco, CA, USA, May 18-21, 2020*. IEEE, 2020, pp. 482–499.
- [16] Z. Chen, F. Zeng, T. Lu, and W. Shu, "Multi-platform application interaction extraction for iot devices," in *25th IEEE International Conference on Parallel and Distributed Systems, ICPADS 2019, Tianjin, China, December 4-6, 2019*. IEEE, 2019, pp. 990–995.
- [17] K. Jiang, H. Zhang, W. Zhang, L. Fang, C. Ge, Y. Yuan, and Z. Liu, "Tapchain: A rule chain recognition model based on multiple features," *Security and Communication Networks*, vol. 2021, no. 1, p. 6568602, 2021.
- [18] K. Hsu, Y. Chiang, and H. Hsiao, "Safechain: Securing trigger-action programming from attack chains," *IEEE Trans. Inf. Forensics Secur.*, vol. 14, no. 10, pp. 2607–2622, 2019.
- [19] E. King, H. Yu, S. Lee, and C. Julien, "Sasha: creative goal-oriented reasoning in smart homes with large language models," *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, vol. 8, no. 1, pp. 1–38, 2024.
- [20] Y. Yang, Y. Wang, S. Sen, L. Li, and Q. Liu, "Unleashing the potential of large language models for predictive tabular tasks in data science," *arXiv preprint arXiv:2403.20208*, 2024.
- [21] OpenAI, "Chatgpt language model," accessed: Oct 2024. [Online]. Available: <https://openai.com/chatgpt>
- [22] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., 2020.
- [23] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., 2022.
- [24] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large language models are zero-shot reasoners," *Advances in neural information processing systems*, vol. 35, pp. 22 199–22 213, 2022.
- [25] L. Yarosh and P. Zave, "Locked or not?: Mental models of iot feature interaction," in *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems, Denver, CO, USA, May 06-11, 2017*, G. Mark, S. R. Fussell, C. Lampe, m. c. schraefel, J. P. Hourcade, C. Appert, and D. Wigdor, Eds. ACM, 2017, pp. 2993–2997.
- [26] W. Brackenburg, A. Deora, J. Ritchey, J. Vallee, W. He, G. Wang, M. L. Littman, and B. Ur, "How users interpret bugs in trigger-action programming," in *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems, CHI 2019, Glasgow, Scotland, UK, May 04-09, 2019*, S. A. Brewster, G. Fitzpatrick, A. L. Cox, and V. Kostakos, Eds. ACM, 2019, p. 552.
- [27] B. Huang, H. Dong, and A. Bouguettaya, "Conflict detection in iot-based smart homes," in *2021 IEEE International Conference on Web Services, ICWS 2021, Chicago, IL, USA, September 5-10, 2021*, C. K. Chang, E. Daminai, J. Fan, P. Ghodous, M. Maximilien, Z. Wang, R. Ward, and J. Zhang, Eds. IEEE, 2021, pp. 303–313.
- [28] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient estimation of word representations in vector space," in *1st International Conference on Learning Representations, ICLR 2013, Scottsdale, Arizona, USA, May 2-4, 2013, Workshop Track Proceedings*, Y. Bengio and Y. LeCun, Eds., 2013.
- [29] Y. Xing, L. Hu, X. Du, Z. Shen, J. Hu, and F. Wang, "Ccdf-tap: A context-aware conflict detection framework for iot trigger-action programming with graph neural network," *IEEE Internet of Things Journal*, vol. 11, no. 19, pp. 31 534–31 544, 2024.
- [30] Y. Xing, L. Hu, X. Du, Z. Shen, J. Hu, and F. Wang, "A privacy-preserving federated graph learning framework for threat detection in iot trigger-action programming," *Expert Systems with Applications*, vol. 255, p. 124724, 2024.
- [31] M. Freestone and S. K. K. Santu, "Word embeddings revisited: Do llms offer something new?" *arXiv preprint arXiv:2402.11094*, 2024.
- [32] F. Corno, L. D. Russis, and A. M. Roffarello, "From users' intentions to IF-THEN rules in the internet of things," *ACM Trans. Inf. Syst.*, vol. 39, no. 4, pp. 53:1–53:33, 2021.
- [33] I. N. B. Yusuf, D. B. A. Jamal, L. Jiang, and D. Lo, "Recipegen++: an automated trigger action programs generator," in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2022, Singapore, Singapore, November 14-18, 2022*, A. Roychoudhury, C. Cadar, and M. Kim, Eds. ACM, 2022, pp. 1672–1676.
- [34] C. Quirk, R. J. Mooney, and M. Galley, "Language to code: Learning semantic parsers for if-this-then-that recipes," in *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing of the Asian Federation of Natural Language Processing, ACL 2015, July*

- 26-31, 2015, Beijing, China, Volume 1: Long Papers. The Association for Computer Linguistics, 2015, pp. 878-888.
- [35] F. Corno, L. D. Russis, and A. M. Roffarello, "Taprec: supporting the composition of trigger-action rules through dynamic recommendations," in *IUI '20: 25th International Conference on Intelligent User Interfaces*, Cagliari, Italy, March 17-20, 2020, F. Paternò, N. Oliver, C. Conati, L. D. Spano, and N. Tintarev, Eds. ACM, 2020, pp. 579-588.
- [36] T. K. Huang, A. Azaria, O. J. Romero, and J. P. Bigham, "Instructable-crowd: Creating IF-THEN rules for smartphones via conversations with the crowd," *Hum. Comput.*, vol. 6, pp. 113-146, 2019.
- [37] F. Corno, L. D. Russis, and A. M. Roffarello, "Recrules: Recommending IF-THEN rules for end-user development," *ACM Trans. Intell. Syst. Technol.*, vol. 10, no. 5, pp. 58:1-58:27, 2019.
- [38] L. Zhang, W. He, O. Morkved, V. Zhao, M. L. Littman, S. Lu, and B. Ur, "Trace2tap: Synthesizing trigger-action programs from traces of behavior," *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.*, vol. 4, no. 3, pp. 104:1-104:26, 2020.
- [39] V. Srinivasan, C. Koehler, and H. Jin, "Ruleselector: Selecting conditional action rules from user behavior patterns," *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.*, vol. 2, no. 1, pp. 35:1-35:34, 2018.
- [40] Y. Cheng, M. Xu, Y. Zhang, K. Li, R. Wang, and L. Yang, "Autoiot: Automated iot platform using large language models," *IEEE Internet of Things Journal*, 2024.
- [41] G. Cimino, V. Deufemia, and M. Limone, "Iot rule generation with cross-view contrastive learning and perplexity-based ranking," *IEEE Internet of Things Journal*, 2025.
- [42] Claude, "Claude," accessed: Oct 2024. [Online]. Available: <https://claude.ai/>
- [43] G. Wu, L. Hu, Y. Hu, X. Xiong, and F. Wang, "Llm4tap: Llm-enhanced tap rule recommendation," *IEEE Internet of Things Journal*, 2025.
- [44] A. Antunes, J. Campos, M. Guimarães, J. Dias, and P. A. Santos, "Prompting for socially intelligent agents with chatgpt," in *Proceedings of the 23rd ACM International Conference on Intelligent Virtual Agents*, 2023, pp. 1-9.
- [45] G. Li, H. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, "Camel: Communicative agents for" mind" exploration of large language model society," *Advances in Neural Information Processing Systems*, vol. 36, pp. 51 991-52 008, 2023.
- [46] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin et al., "Metagpt: Meta programming for a multi-agent collaborative framework," in *The Twelfth International Conference on Learning Representations*, 2023.
- [47] D. Rivkin, F. Hogan, A. Feriani, A. Konar, A. Sigal, X. Liu, and G. Dudek, "Aiot smart home via autonomous llm agents," *IEEE Internet of Things Journal*, 2024.
- [48] S. Kouah, A. Saighi, M. Ammi, A. Naït Si Mohand, M. I. Kouah, and D. Megias, "Internet of things-based multi-agent system for the control of smart street lighting," *Electronics*, vol. 13, no. 18, p. 3673, 2024.
- [49] W. Zhang, Y. Meng, Y. Liu, X. Zhang, Y. Zhang, and H. Zhu, "Homonit: Monitoring smart home apps from encrypted traffic," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS 2018, Toronto, ON, Canada, October 15-19, 2018*, D. Lie, M. Mannan, M. Backes, and X. Wang, Eds. ACM, 2018, pp. 1074-1088.
- [50] T. Gu, Z. Fang, A. Abhishek, H. Fu, P. Hu, and P. Mohapatra, "Iotgaze: Iot security enforcement via wireless context analysis," in *39th IEEE Conference on Computer Communications, INFOCOM 2020, Toronto, ON, Canada, July 6-9, 2020*. IEEE, 2020, pp. 884-893.
- [51] C. Fu, Q. Zeng, and X. Du, "Hawatcher: Semantics-aware anomaly detection for appified smart homes," in *30th USENIX Security Symposium, USENIX Security 2021, August 11-13, 2021*, M. D. Bailey and R. Greenstadt, Eds. USENIX Association, 2021, pp. 4223-4240.
- [52] H. Zheng and H. Hu, "Missile: A system of mobile inertial sensor-based sensitive indoor location eavesdropping," *IEEE Transactions on Information Forensics and Security*, vol. 15, pp. 3137-3151, 2020.
- [53] R. Li, H. Hu, and Q. Ye, "Rfrack: Stealthy location inference and tracking attack on wi-fi devices," *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 5925-5939, 2024.
- [54] Z. Chen, H. Hu, and J. Yu, "Privacy-preserving large-scale location monitoring using bluetooth low energy," in *11th International Conference on Mobile Ad-hoc and Sensor Networks, MSN 2015, Shenzhen, China, December 16-18, 2015*. IEEE Computer Society, 2015, pp. 69-78.
- [55] H. P. Li, H. Hu, and J. Xu, "Nearby friend alert: Location anonymity in mobile geosocial networks," *IEEE Pervasive Computing*, vol. 12, no. 4, pp. 62-70, 2013.
- [56] X. Lin, H. Hu, H. P. Li, J. Xu, and B. Choi, "Private proximity detection and monitoring with vicinity regions," in *Proceedings of the 12th International ACM Workshop on Data Engineering for Wireless and Mobile Access*, ser. MobiDE '13. New York, NY, USA: Association for Computing Machinery, 2013, p. 5-12.
- [57] Blakadder, "Tasmota templates." [Online]. Available: <https://templates.blakadder.com/search.html>
- [58] S. Tonmoy, S. Zaman, V. Jain, A. Rani, V. Rawte, A. Chadha, and A. Das, "A comprehensive survey of hallucination mitigation techniques in large language models," *arXiv preprint arXiv:2401.01313*, 2024.
- [59] G. Cimino and V. Deufemia, "Sigfrid: unsupervised, platform-agnostic interference detection in iot automation rules," *ACM Transactions on Internet of Things*, vol. 6, no. 2, pp. 1-33, 2025.
- [60] Prolific, "Prolific website," accessed: Oct 2024. [Online]. Available: <https://www.prolific.com/>
- [61] K. Arkoudas, "Gpt-4 can't reason," *arXiv preprint arXiv:2308.03762*, 2023.
- [62] S. Dhingra, M. Singh, S. Vaisakh, N. Malviya, and S. S. Gill, "Mind meets machine: Unravelling gpt-4's cognitive psychology," *BenchCouncil Transactions on Benchmarks, Standards and Evaluations*, vol. 3, no. 3, p. 100139, 2023.
- [63] Anysphere, "Cursor: The ai code editor," 2025. [Online]. Available: <https://cursor.com>
- [64] Anthropic, "Claude code," 2025, agentic coding tool for terminal workflows. [Online]. Available: <https://code.claude.com>
- [65] F. James, "Monte carlo theory and practice," *Reports on Progress in Physics*, vol. 43, no. 9, p. 1145, 1980.